

DBMS

Database Management Systems — Complete Notes

Section	Topic
1	Introduction to DBMS
2	DBMS Architecture
3	Data Models
4	Entity-Relationship (ER) Model
5	Relational Model & Keys
6	Normalization
7	SQL Basics
8	Joins
9	Transactions & ACID Properties
10	Concurrency Control
11	Indexing
12	Database Recovery
13	SQL vs NoSQL

1. Introduction to DBMS

A Database Management System (DBMS) is software that enables users to define, create, maintain, and control access to a database. It acts as an interface between the database and end users or application programs.

Advantages of DBMS over File Systems

- Reduces data redundancy and inconsistency
- Provides data security and access control
- Supports concurrent access by multiple users
- Ensures data integrity through constraints
- Provides backup and recovery mechanisms

Types of DBMS

- Hierarchical DBMS — data organized in a tree-like structure
- Network DBMS — data organized as a graph with multiple parent-child links
- Relational DBMS (RDBMS) — data organized into tables (e.g. MySQL, PostgreSQL, Oracle)
- Object-Oriented DBMS — data stored as objects (e.g. ObjectDB)

2. DBMS Architecture

Three-Schema Architecture

- External Level — how individual users view the data (views)
- Conceptual Level — describes what data is stored and relationships (logical structure)
- Internal Level — describes how data is physically stored (storage structure)

1-Tier, 2-Tier & 3-Tier Architecture

- 1-Tier — database directly accessible to the user (e.g. local database)
- 2-Tier — client communicates directly with the database server
- 3-Tier — client, application server, and database server are separate layers (most common for web apps)

3. Data Models

A data model defines how data is logically structured, stored, and manipulated.

Model	Description
Hierarchical	Tree structure; one parent, many children

Network	Graph structure; supports many-to-many relationships
Relational	Data in tables (rows and columns) linked by keys
Object-Oriented	Data represented as objects, like in OOP
ER Model	Conceptual model using entities and relationships

4. Entity-Relationship (ER) Model

The ER Model represents the database structure using entities, attributes, and relationships, typically shown in an ER diagram.

Key Terms

- Entity — a real-world object (e.g. Student, Employee)
- Attribute — a property of an entity (e.g. Name, Age)
- Relationship — an association between two or more entities (e.g. 'enrolls in')
- Entity Set — a collection of similar entities

Types of Attributes

- Simple — cannot be divided further (e.g. Age)
- Composite — can be divided into sub-parts (e.g. Name -> First, Last)
- Derived — can be derived from other attributes (e.g. Age from Date of Birth)
- Multivalued — can hold multiple values (e.g. Phone Numbers)

Types of Relationships (Cardinality)

- One-to-One (1:1) — e.g. a person has one passport
- One-to-Many (1:N) — e.g. one department has many employees
- Many-to-Many (M:N) — e.g. students enroll in many courses, courses have many students

5. Relational Model & Keys

In the relational model, data is organized into tables (relations), where each row is a tuple and each column is an attribute.

Key Type	Description
Primary Key	Uniquely identifies each record in a table
Candidate Key	A column (or set) that could qualify as a primary key
Foreign Key	References the primary key of another table
Composite Key	A key made up of two or more columns
Super Key	A set of attributes that can uniquely identify a tuple
Alternate Key	A candidate key not chosen as the primary key

6. Normalization

Normalization is the process of organizing data to reduce redundancy and avoid anomalies (insertion, update, deletion) by dividing tables into smaller, related tables.

Normal Form	Rule
1NF	Each column contains atomic (indivisible) values; no repeating groups
2NF	1NF + no partial dependency (non-key attributes depend on the whole key)
3NF	2NF + no transitive dependency (non-key attributes depend only on the key)
BCNF	Stricter 3NF; every determinant must be a candidate key

Normalization reduces redundancy but can increase the number of joins needed for queries — denormalization is sometimes used intentionally to improve read performance.

7. SQL Basics

Categories of SQL Commands

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP	Define database structure
DML	SELECT, INSERT, UPDATE, DELETE	Manipulate data
DCL	GRANT, REVOKE	Control access permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

Common SQL Queries

```
CREATE TABLE Students (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  age INT  
);  
  
INSERT INTO Students VALUES (1, 'Alice', 20);  
  
SELECT * FROM Students WHERE age > 18;  
  
UPDATE Students SET age = 21 WHERE id = 1;  
  
DELETE FROM Students WHERE id = 1;
```

Aggregate Functions & Grouping

```
SELECT department, COUNT(*), AVG(salary)
FROM Employees
GROUP BY department
HAVING COUNT(*) > 5;
```

8. Joins

Joins are used to combine rows from two or more tables based on a related column.

Join Type	Description
INNER JOIN	Returns matching rows from both tables
LEFT JOIN	All rows from left table + matched rows from right
RIGHT JOIN	All rows from right table + matched rows from left
FULL OUTER JOIN	All rows from both tables, matched where possible
SELF JOIN	Table joined with itself
CROSS JOIN	Cartesian product of both tables

```
SELECT Students.name, Courses.course_name
FROM Students
INNER JOIN Courses ON Students.id = Courses.student_id;
```

9. Transactions & ACID Properties

A transaction is a sequence of operations performed as a single logical unit of work. ACID properties ensure transactions are processed reliably.

Property	Meaning
Atomicity	Transaction is all-or-nothing
Consistency	Database moves from one valid state to another
Isolation	Concurrent transactions don't interfere with each other
Durability	Once committed, changes persist even after a system failure

```
BEGIN TRANSACTION;
    UPDATE Accounts SET balance = balance - 500 WHERE id = 1;
    UPDATE Accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;
```

10. Concurrency Control

Concurrency control ensures correct results when multiple transactions execute simultaneously on shared data.

Common Problems

- Dirty Read — reading uncommitted data from another transaction
- Lost Update — one transaction's update is overwritten by another

- Non-Repeatable Read — a row's value changes between two reads in the same transaction
- Phantom Read — new rows appear in a repeated query within the same transaction

Concurrency Control Techniques

- Lock-based protocols (shared/exclusive locks)
- Timestamp ordering protocol
- Optimistic concurrency control

11. Indexing

An index is a data structure that improves the speed of data retrieval operations on a table, at the cost of additional storage and slower writes.

Index Type	Description
Primary Index	Built automatically on the primary key
Clustered Index	Physically reorders table rows to match the index (one per table)
Non-Clustered Index	Separate structure pointing to actual row locations
Composite Index	Index built on multiple columns
Unique Index	Ensures all values in the indexed column are distinct

```
CREATE INDEX idx_name ON Students(name);
```

12. Database Recovery

Recovery techniques restore the database to a consistent state after a failure (system crash, power loss, etc.).

- Log-based recovery — maintains logs of all transactions for redo/undo operations
- Checkpointing — periodically saves the database state to reduce recovery time
- Shadow paging — maintains two page tables (current and shadow) for recovery
- Backup — regular full/incremental backups protect against data loss

13. SQL vs NoSQL

Aspect	SQL (RDBMS)	NoSQL
Schema	Fixed, predefined schema	Dynamic/flexible schema
Data Structure	Tables (rows & columns)	Documents, key-value, graph, columnar
Scalability	Vertical scaling	Horizontal scaling
Transactions	Strong ACID compliance	Often BASE (eventual consistency)
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis

End of Notes — DBMS concepts form the backbone of nearly all software systems. Practice writing SQL queries and drawing ER diagrams to strengthen your understanding.